EL

ఒక్క PROBLEM · మొదటి పంక్తి నుంచి పూర్తి SYSTEM వరకు

Design an Elevator System

> ఇది ఒక పూర్తి design ని చూపించే doc కాదు. ఇది మనం **కలిసి** ఒక lift system ని కట్టే doc – ఒక్కో పంక్తి, ఒక్కో కారణం, ఒక్కో తప్పుతో సహా.

ప్రతి code ముక్కకి ముందు **"ఇది ఎందుకు కావాలి"**, తర్వాత **"ఇది ఏం చేస్తుంది"**. మధ్యలో మన design రెండుసార్లు విరిగిపోతుంది – ఆ విరిగిన output ని కూడా చూపిస్తాను, ఎందుకంటే *ఎందుకు విరిగిందో* తెలియకుండా *ఎలా సరిచేయాలో* గుర్తుండదు.

తెలుగు + ENGLISH

YASWANTH · DEEP DIVE 01

ఈ doc ఎలా చదవాలి

ఇది చదవడానికి ఉద్దేశించినది కాదు – **పక్కన ఒక editor తెరిచి, కలిసి రాయడానికి**.

ప్రతి దశలో ఒక చిన్న code ముక్క ఉంటుంది. దాన్ని మీరే type చేసి, `node` lo run చేసి, ఇక్కడ చూపించిన output వస్తుందో లేదో చూడండి. ఈ doc lo ఉన్న **ప్రతి output నిజంగా run చేసి తీసినదే** – నేను ఊహించి రాసినది ఒక్కటి కూడా లేదు. మీకు వేరే output వస్తే, అక్కడ ఏదో తేడా ఉంది, అది వెతకడమే అసలు నేర్చుకోవడం.

> **ఈ DOC మిగతా LLD BOOK కంటే ఎందుకు వేరు**
>
> `LLD_Design_Problems_Telugu.pdf` lo ఈ problem ఒక **interview నమూనా** గా ఉంటుంది – 45 నిమిషాల్లో ఏం చెప్పాలో, ఏ క్రమంలో చెప్పాలో. అది సరైనదే, కానీ అక్కడ code ఒకేసారి పూర్తిగా కనిపిస్తుంది.
>
> ఇక్కడ దానికి **పూర్తి వ్యతిరేకం** చేస్తున్నాం. ఇక్కడ code **పెరుగుతుంది**. ఐదు lines తో మొదలై, ప్రతి దశలో ఒక కొత్త అవసరం వచ్చి, దాన్ని తీర్చడానికి మరో ఐదు lines చేరతాయి. చివరికి అదే పూర్తి class అవుతుంది – కానీ అప్పటికి మీరు **ప్రతి పంక్తిని మీరే రాశారు**, మరియు ప్రతి పంక్తి ఎందుకు ఉందో మీకు తెలుసు.
>
> **క్రమం:** ముందు ఇది (అర్థం చేసుకోవడానికి), తర్వాత ఆ book (interview lo చెప్పడానికి).

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Interviewer అడిగింది ఏమిటి – మరియు అందులో దాగినవి
2. మనసులో ఒకసారి lift నడిపి చూద్దాం
3. రెండు రకాల requests – ఇదే ఈ problem యొక్క గుండె
4. Clarifying questions – మరియు ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – ఒకే lift, మొదటి version

5. Step – ఒక lift కి ఏమి తెలియాలి?
6. Step – దాన్ని ఒక అడుగు కదిలించడం
7. Step – నడిపి చూద్దాం, మరియు **మొదటి విరుపు**
8. Step – పలు stops ని గుర్తుపెట్టుకోవడం

Part 3 – దిక్కు ఎంపిక – ఇక్కడే అసలు కథ

9. Step – "దగ్గరలో ఉన్నది ముందు" – సహజమైన ఆలోచన
10. **రెండో విరుపు** – starvation ని కళ్ళతో చూడటం
11. Step – LOOK algorithm – మరియు అదే traffic మీద ఫలితం
12. Step – తలుపు, మరియు అది ఒక tick ఎందుకు తినాలి

Part 4 – ఒక lift పూర్తి

13. పూర్తి `Elevator` class – ఇప్పుడు ప్రతి పంక్తీ మీకు తెలుసు
14. Invariants – ఎప్పుడూ నిజమై ఉండాల్సినవి

Part 5 – ఒకటి నుంచి నాలుగు lifts కి

15. కొత్త సమస్య – ఈ call ని ఎవరికి ఇవ్వాలి?
16. Step – cost function ని ఒక్కో పదం చొప్పున కట్టడం
17. Step – దాన్ని Strategy గా బయట పెట్టడం, మరియు ఎందుకు
18. Step – `ElevatorSystem` – రెండు call రకాలు ఇక్కడ కలుస్తాయి

Part 6 – పూర్తి system

19. మొత్తం code ఒకే చోట
20. ఒక పూర్తి simulation – tick by tick

Part 7 – Interview lo

21. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
22. నోటితో చెప్పాల్సిన English script
23. Follow-ups – మరియు మన design ఎలా తట్టుకుంటుంది
24. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

> కోడ్ రాయడం కంటే ముందు, సమస్య ఆకారం తెలియాలి. ఈ Part మొత్తం ఒక్క పంక్తి code కూడా లేకుండా ఉంటుంది – మరియు అదే ఉద్దేశం.

---

1. Interviewer అడిగింది ఏమిటి – మరియు అందులో దాగినవి

Interviewer ఒక్క వాక్యం చెప్తాడు:

> *"Design the control system for a building with four elevators and fifteen floors."*

ఈ వాక్యం చిన్నది, కానీ దీని వెనక మూడు వేర్వేరు సమస్యలు దాగి ఉన్నాయి. వాటిని విడదీయకపోతే మనం గందరగోళంగా code రాస్తాం.

మొదటి సమస్య – ఒక lift తనని తాను ఎలా నడుపుకుంటుంది? ఇది 5వ అంతస్తులో ఉంది, 9కి వెళ్ళాలి, మధ్యలో 7 కూడా ఆగాలి. ఏ క్రమంలో ఆగాలి? ఎప్పుడు దిక్కు మార్చాలి? ఇది **ఒక lift లోపలి సమస్య**.

రెండో సమస్య – ఒక కొత్త request ఏ lift కి ఇవ్వాలి? 3వ అంతస్తులో ఒకరు పైకి వెళ్ళాలని నొక్కారు. నాలుగు lifts ఉన్నాయి. ఎవరికి ఇవ్వాలి? ఇది **lifts మధ్య సమస్య** – పూర్తిగా వేరే స్వభావం ఉన్నది.

మూడో సమస్య – ఇవి రెండూ ఒకదానికొకటి తెలియకుండా ఎలా ఉంచాలి? అంటే, రేపు scheduling నియమం మార్చాలనుకుంటే lift code ని ముట్టుకోకూడదు.

ఈ మూడు విభజనని **interview మొదట్లోనే బయటికి చెప్పండి**. "ఇందులో నాకు రెండు వేర్వేరు సమస్యలు కనిపిస్తున్నాయి – ఒక lift తనని తాను నడుపుకోవడం, మరియు requests ని lifts కి కేటాయించడం. నేను ముందు మొదటిది చేసి, తర్వాత రెండోది చేస్తాను." – ఈ ఒక్క వాక్యం మీ మొత్తం interview ని ఒక స్పష్టమైన దారిలో పెడుతుంది.

---

2. మనసులో ఒకసారి lift నడిపి చూద్దాం

Code రాయడానికి ముందు, ఒక నిజమైన దృశ్యాన్ని నెమ్మదిగా ఊహించుకుందాం. ఇది వృథా కాదు – ఇక్కడ దొరికే ప్రశ్నలే మన class design ని నిర్ణయిస్తాయి.

మీరు **5వ అంతస్తులో** నిలబడి ఉన్నారు. **పైకి** వెళ్ళాలి. గోడ మీద ఉన్న ▲ బటన్ నొక్కారు.

ఇప్పుడు ఆలోచించండి – ఆ క్షణంలో system కి ఏమి తెలుసు?

- మీరు **5వ అంతస్తులో** ఉన్నారని తెలుసు.
- మీరు **పైకి** వెళ్ళాలని తెలుసు.
- మీరు **ఎన్నో అంతస్తుకి** వెళ్ళాలో – **తెలియదు**. మీరు ఇంకా చెప్పలేదు!

ఇప్పుడు lift వచ్చింది, మీరు ఎక్కారు, లోపల **9** నొక్కారు. ఇప్పుడు system కి ఏమి తెలుసు?

- ఈ నిర్దిష్ట lift **9కి వెళ్ళాలని** తెలుసు.
- ఏ lift అనేది **ఇప్పటికే నిర్ణయమైపోయింది** – మీరు ఇప్పటికే అందులో ఉన్నారు.

> **ఇదే ఈ PROBLEM LO 90% మంది మిస్ చేసే విషయం**
>
> ఆ రెండు బటన్లు – బయటి ▲ మరియు లోపలి **9** – **ఒకే రకమైన request కాదు**. అవి వేరే సమాచారం మోసుకొస్తాయి, మరియు వాటిని వేరే విధంగా నిర్వహించాలి.
>
> చాలా మంది వెంటనే ఒక `Request { floor, direction }` class రాసేస్తారు. తర్వాత scheduling రాసేటప్పుడు "ఇక్కడ direction ఎందుకు లేదు?" అని తడబడతారు. ఎందుకంటే **లోపలి request కి direction అవసరం లేదు** – destination తెలుసు కదా.

---

3. రెండు రకాల requests – ఇదే ఈ problem యొక్క గుండె

పైన చూసినదాన్ని ఇప్పుడు స్పష్టంగా పేరుపెట్టి విడదీద్దాం. ఈ రెండు పేర్లు నిజమైన lift industry పరిభాష – interview lo వీటిని వాడితే మీరు homework చేశారని తెలుస్తుంది.

HALL CALL VS CAR CALL · ఒకే SYSTEM, రెండు పూర్తిగా వేరే REQUESTS

ఈ రెండిటి తేడా తెలియకపోతే scheduling code గందరగోళం అవుతుంది

HALL CALL – బయటి బటన్

"5వ అంతస్తులో ఎవరో పైకి వెళ్ళాలి"

తెలిసినవి: floor = 5, direction = UP

తెలియనిది: destination (ఇంకా చెప్పలేదు)

ఏ lift వెళ్ళాలి? – **system నిర్ణయిస్తుంది**

ఇక్కడే scheduling అవసరం వస్తుంది

CAR CALL – లోపలి బటన్

"ఈ lift lo ఎవరో 9 నొక్కారు"

తెలిసినవి: destination = 9

direction? – అవసరం లేదు, గమ్యం తెలుసు

ఏ lift? – ఇప్పటికే నిర్ణయమైంది

ఇక్కడ scheduling అవసరమే లేదు

దీని నుంచి వచ్చే API – ఇది మీరే derive చేయాలి, గుర్తుపెట్టుకోకూడదు

hallCall(floor, direction) → ఏ lift వెళ్ళిందో తిరిగి చెప్తుంది · **carCall(liftId, floor)** → ఏమీ తిరిగి ఇవ్వదు
రెండింటి signature వేరు – ఎందుకంటే అవి తెలిసిన సమాచారం వేరు

ఇక్కడ ఒక సాధారణ తప్పు

ఒకే **Request** class రాసి, car call కి direction ని **null** పెట్టడం. అది పనిచేస్తుంది కానీ ప్రతి చోటా null check కావాలి – ఆ null మీ design lo ఒక అబద్ధం.

ఒక్క వాక్యంలో చెప్పాలంటే:

> **Hall call** = "ఎవరో నన్ను పిలుస్తున్నారు, ఎవరు వెళ్ళాలో నిర్ణయించు." **Car call** = "నేను ఇక్కడికి వెళ్ళాలి."

మొదటిదానికి **తెలివి** కావాలి. రెండోదానికి **జ్ఞాపకం** చాలు.

---

4. Clarifying questions – మరియు ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ఇప్పుడు interviewer ని ప్రశ్నలు అడుగుదాం. కానీ ఒక ముఖ్యమైన విషయం: **ప్రతి ప్రశ్నకీ ఒక ఉద్దేశం ఉండాలి.** "ఎన్ని users?" అని అడిగి, జవాబుని ఎక్కడా వాడకపోతే – అది ఒక ritual, ఒక ప్రశ్న కాదు. Interviewer ఆ తేడాని గమనిస్తాడు.

ప్రశ్న	దీని జవాబు నా design ని ఎలా మారుస్తుంది
ఎన్ని lifts, ఎన్ని floors?	ఒకే lift అయితే scheduling అనే భాగమే అవసరం లేదు. 4 lifts అంటే §15-17 మొత్తం అవసరం
దేన్ని optimise చేయాలి – wait time నా, energy నా?	ఇదే cost function ని నిర్ణయిస్తుంది (§16). Energy అయితే lifts ని కదిలించకపోవడమే మంచిది; wait time అయితే వ్యతిరేకం
Capacity limit ఉందా?	ఉంటే "నిండిన lift కి hall call ఇవ్వకూడదు" – cost lo ఒక `Infinity` (§16)
Express lifts (కొన్ని floors మాత్రమే) ఉన్నాయా?	ఉంటే ప్రతి lift కి `servableFloors` కావాలి. లేకపోతే ఆ complexity వద్దు – **YAGNI**
Emergency / maintenance mode?	ఉంటే state machine lo అదనపు states, మరియు "ఈ lift ఇప్పుడు available లేదు" అనే భావన
Real-time నా, simulation నా?	ఇది **అతి ముఖ్యమైన ప్రశ్న** – కింద వివరంగా

"Tick-based నా?" – ఈ ప్రశ్న ఎందుకు అంత ముఖ్యం

మీరు ఇలా అడగాలి: *"Time ని నేను ఎలా model చేయాలి? నిజమైన గడియారం ప్రకారం (setInterval, async), లేక ఒక `step()` method ని మనం పిలిచినప్పుడల్లా ఒక అడుగు కదిలేలా?"*

Interviewer దాదాపు ఎప్పుడూ రెండోది చెప్తాడు. కారణం:

- **Test చేయడం సులభం.** `step()` ని 10 సార్లు పిలిచి, floor ఎక్కడ ఉందో చూడొచ్చు. Timers తో test రాయాలంటే అది ఒక పీడకల.
- **Debug చేయడం సులభం.** ఏ tick lo ఏమి జరిగిందో కచ్చితంగా చూడొచ్చు.
- **నిజమైన system కి కూడా ఇదే సరైనది** – పైన ఒక timer పెట్టి ప్రతి సెకనుకి `step()` పిలిస్తే చాలు. అంటే **time ని బయట పెట్టాం**.

> **ఇది ఒక పెద్ద DESIGN సూత్రం – ఇక్కడ చిన్నగా కనిపిస్తోంది**
>
> "Time ని inject చేయడం" అనేది ఒక universal ఆలోచన. Rate limiter, cache TTL, scheduler, session timeout – వీటన్నిటిలోనూ ఇదే వస్తుంది. Class లోపల `Date.now()` పిలిస్తే ఆ class ని test చేయడం కష్టం; సమయాన్ని బయట నుంచి ఇస్తే అది సులభం.
>
> మన lift lo `step()` అంటేనే "ఒక యూనిట్ సమయం గడిచింది". గడియారం ఎవరిది అనేది lift కి తెలియదు – అదే సరైనది.

Part 2 – ఒకే lift, మొదటి version

> ఇప్పుడు code మొదలు. కానీ నాలుగు lifts గురించి ఇప్పుడే ఆలోచించొద్దు – **ఒక్క lift** ని సరిగ్గా నడిపించడం నేర్చుకుందాం. అది వచ్చాక నాలుగు lifts అనేది ఒక చిన్న పొర మాత్రమే.

---

5. Step – ఒక lift కి ఏమి తెలియాలి?

ఒక class రాయడం మొదలుపెట్టేటప్పుడు అడగాల్సిన మొదటి ప్రశ్న ఇది: **"ఈ object తన గురించి ఏమి గుర్తుపెట్టుకోవాలి?"**

ఒక lift ని ఊహించుకోండి. అది ఈ క్షణంలో తన గురించి ఏమి తెలిసి ఉండాలి?

- **నేను ఇప్పుడు ఏ అంతస్తులో ఉన్నాను?** – ఇది స్పష్టం.
- **నేను ఎక్కడికి వెళ్తున్నాను?** – ఇది కూడా అవసరం, లేకపోతే కదలడం ఎలా?

మొదలుపెట్టడానికి ఇంతే చాలు. `direction`, `doorOpen`, `capacity` – ఇవన్నీ నిజమే, కానీ **ఇప్పుడు అవసరం లేదు**. అవసరం వచ్చినప్పుడు చేరుస్తాం. ఇదే మంచి పద్ధతి: ఖాళీ fields తో మొదలుపెట్టి వాటిని నింపడానికి ప్రయత్నించడం కంటే, అవసరం నుంచి fields ని పుట్టించడం మేలు.

```
class Elevator {
  constructor(id) {
    this.id = id;
    this.currentFloor = 0;     // ఇప్పుడు ఎక్కడ ఉన్నాను
    this.target = null;        // ఎక్కడికి వెళ్ళాలి (null = ఎక్కడికీ లేదు)
  }

  goTo(floor) {
    this.target = floor;
  }
}
```

ఇది ఏం చేస్తుంది: ఒక lift ని సృష్టిస్తుంది, అది 0వ అంతస్తులో ఖాళీగా నిలబడి ఉంటుంది. `goTo(3)` పిలిస్తే "3కి వెళ్ళాలి" అని గుర్తుపెట్టుకుంటుంది.

ఇది ఇంకా ఏం చేయదు: కదలదు. అది తర్వాతి దశ.

`id` ఎందుకు? ఇప్పుడు ఒకే lift ఉంది కాబట్టి అనవసరం అనిపించొచ్చు. కానీ §15 lo నాలుగు lifts వచ్చినప్పుడు – "ఏ lift వచ్చింది?" అని చెప్పడానికి ఇది కావాలి. ఇది ముందుచూపు కాదు; lifts కి పేర్లు ఉండటం సహజం (నిజ భవనాల్లో కూడా "Lift A", "Lift B" అని రాసి ఉంటుంది).

---

6. Step – దాన్ని ఒక అడుగు కదిలించడం

ఇప్పుడు కదలిక. గుర్తుందా – §4 lo మనం **tick-based** అని నిర్ణయించుకున్నాం. అంటే ఒక `step()` method, మరియు ప్రతి పిలుపు = ఒక అంతస్తు కదలిక.

ఆలోచన చాలా సులభం: *గమ్యం నా కంటే పైన ఉంటే ఒక అంతస్తు పైకి. కింద ఉంటే ఒక అంతస్తు కిందికి. చేరుకున్నాక గమ్యాన్ని మర్చిపో.*

```
step() {
  if (this.target === null) return;              // పని లేదు, ఆగిపో

  if (this.currentFloor < this.target) this.currentFloor++;
  else if (this.currentFloor > this.target) this.currentFloor--;
  else this.target = null;                       // చేరాం — గమ్యం ఖాళీ
}
```

ఇది ఏం చేస్తుంది: ఒక్కో పిలుపుకి ఒక్కో అంతస్తు కదులుతుంది. గమ్యం చేరాక `target` ని `null` చేస్తుంది, తద్వారా తర్వాతి `step()` ఏమీ చేయదు.

ఒక subtle విషయం గమనించండి: చేరుకున్న అదే tick lo మనం `target` ని ఖాళీ చేయట్లేదు – ఒక tick ఆలస్యంగా చేస్తున్నాం. అంటే lift గమ్యం చేరిన tick lo `currentFloor` మారుతుంది, ఆ తర్వాతి tick lo `target` `null` అవుతుంది. ఇది ఇప్పుడు హానికరం కాదు, కానీ §12 lo తలుపు వచ్చినప్పుడు ఈ "ఒక tick" ఆలోచనే మనకి ఉపయోగపడుతుంది.

---

7. Step – నడిపి చూద్దాం, మరియు మొదటి విరుపు

రాసినది పనిచేస్తుందో లేదో చూడకుండా ముందుకు వెళ్ళకూడదు. నడిపి చూద్దాం:

```
const e = new Elevator('A');
e.goTo(3);
for (let t = 1; t <= 5; t++) {
  e.step();
  console.log(`tick ${t}: floor ${e.currentFloor}`);
}
```

```
tick 1: floor 1
tick 2: floor 2
tick 3: floor 3
tick 4: floor 3
tick 5: floor 3
```

పనిచేస్తోంది. 3కి చేరి ఆగిపోయింది. **కానీ** – ఇప్పుడు ఒక నిజమైన పరిస్థితిని పెడదాం. Lift 3కి వెళ్తున్న మధ్యలో, ఇంకొకరు లోపల **7** నొక్కారు:

```
const e = new Elevator('A');
e.goTo(3);
e.step();  console.log(`tick 1: floor ${e.currentFloor}, target ${e.target}`);

e.goTo(7);                                 // ← ఇంకొకరు 7 నొక్కారు
e.step();  console.log(`tick 2: floor ${e.currentFloor}, target ${e.target}`);
e.step();  console.log(`tick 3: floor ${e.currentFloor}, target ${e.target}`);

console.log('3వ అంతస్తు వాళ్ళ సంగతి?', e.target);
```

```
tick 1: floor 1, target 3
tick 2: floor 2, target 7
tick 3: floor 3, target 7
3వ అంతస్తు వాళ్ళ సంగతి? 7
```

మొదటి విరుపు – దీన్ని జాగ్రత్తగా చూడండి

Tick 3 lo lift **3వ అంతస్తు గుండా వెళ్ళిపోయింది** – కానీ ఆగలేదు. ఎందుకంటే tick 2 lo `goTo(7)` పిలిచినప్పుడు, అది `target` ని **7 తో తుడిచేసింది**. 3 అనే సమాచారం శాశ్వతంగా పోయింది.

నిజ జీవితంలో ఇది ఎలా కనిపిస్తుంది? మీరు lift lo 3 నొక్కారు. మీ పక్కన ఉన్నవాళ్ళు 7 నొక్కారు. Lift మీ అంతస్తు దాటి పైకి వెళ్ళిపోయింది. **ఇది ఒక bug కాదు – ఇది మన design lo ఒక తప్పు ఊహ.**

తప్పు ఊహ ఏమిటి? – **"ఒక lift కి ఒకే ఒక గమ్యం ఉంటుంది."**

కానీ నిజం ఏమిటంటే, ఒక lift కి **ఒకేసారి పలు గమ్యాలు** ఉంటాయి. అది ఒక *గమ్యం* కాదు, అది ఒక **జాబితా**.

ఇది చిన్న మార్పులా కనిపిస్తుంది. కానీ ఇది `target` అనే ఒక field ని `stops` అనే ఒక collection గా మారుస్తుంది – మరియు దానితో పాటు ఒక పెద్ద కొత్త ప్రశ్న తెస్తుంది: **"ఈ జాబితాలో ఏది ముందు?"** ఆ ప్రశ్నే Part 3 మొత్తం.

---

8. Step – పలు stops ని గుర్తుపెట్టుకోవడం

`target` ని తీసేసి `stops` పెడదాం. ఏ collection వాడాలి?

- **Array?** అయితే duplicate floors చేరతాయి – ఒకే అంతస్తుని ఇద్దరు నొక్కితే రెండుసార్లు వస్తుంది. అది తప్పు.
- **Set?** Duplicates దానంతట అదే పోతాయి, మరియు `has`, `delete` రెండూ O(1). **ఇదే సరైనది.**
- **Priority queue?** ఇది "ఏది ముందు" అనే క్రమాన్ని ఇప్పుడే నిర్ణయించేస్తుంది. కానీ ఆ క్రమం lift యొక్క దిక్కు మీద ఆధారపడుతుంది, మరియు దిక్కు మారుతూ ఉంటుంది. కాబట్టి ఇది **అనవసరమైన complexity**.

> **Interview lo ఈ ఎంపికని బయటికి చెప్పండి.** "నేను Set వాడుతున్నాను ఎందుకంటే ఒకే floor ని రెండుసార్లు నొక్కితే అది ఒక్కసారే ఉండాలి, మరియు నాకు లుక్అప్ O(1) కావాలి. Priority queue కూడా ఆలోచించాను, కానీ priority ఇక్కడ దిక్కుని బట్టి మారుతుంది కాబట్టి అది సరిపోదు." – ఇలాంటి ఒక వాక్యం మీ data structure ఎంపికకి కారణం ఇస్తుంది.

```
const Direction = Object.freeze({ UP: 1, IDLE: 0, DOWN: -1 });

class Elevator {
  constructor(id) {
    this.id = id;
    this.currentFloor = 0;
    this.direction = Direction.IDLE;      // కొత్తది — ఏ దిక్కు వెళ్తున్నాం
    this.stops = new Set();               // target స్థానంలో ఇది
  }

  addStop(floor) {
    if (floor !== this.currentFloor) this.stops.add(floor);
  }
}
```

`Direction` ఎందుకు వచ్చింది, మరియు అది `1 / 0 / -1` ఎందుకు

`stops` ఒక Set అయ్యాక, "తర్వాత ఎక్కడికి?" అనేది ఇక స్పష్టం కాదు. దానికి lift ఏ **దిక్కు** వెళ్తోందో తెలియాలి. కాబట్టి `direction` ఒక కొత్త అవసరంగా పుట్టింది – నేను ముందే ప్లాన్ చేసి పెట్టలేదు.

విలువలు `UP: 1, IDLE: 0, DOWN: -1` ఎందుకంటే – వీటితో కదలిక **ఒక్క పంక్తి** అవుతుంది:

```
this.currentFloor += this.direction;   // UP అయితే +1, DOWN అయితే -1, IDLE అయితే
కదలదు
```

`'UP' / 'DOWN'` అని strings వాడితే ఇక్కడ ఒక `if/else` రాయాల్సి వచ్చేది. ఈ చిన్న ఎంపిక తర్వాత మూడు చోట్ల code ని శుభ్రంగా ఉంచుతుంది. Interview lo ఇది చెప్తే చిన్న విషయమే, కానీ మంచి సంకేతం.

`Object.freeze` ఎందుకు? `Direction.UP = 5` అని ఎవరైనా పొరపాటున రాస్తే అది నిశ్శబ్దంగా పనిచేస్తుంది, మరియు మొత్తం system విచిత్రంగా ప్రవర్తిస్తుంది. `freeze` ఆ తలుపు మూస్తుంది. JavaScript lo నిజమైన enum లేదు కాబట్టి ఇదే దానికి దగ్గరి రూపం.

Part 3 – దిక్కు ఎంపిక – ఇక్కడే అసలు కథ

> ఈ భాగమే ఈ problem యొక్క నిజమైన కేంద్రం. `stops` ఒక Set అయ్యాక, "తర్వాత ఎక్కడికి?" అనే ప్రశ్నకి జవాబు చెప్పాలి. ఆ జవాబు మీద lift బాగా ప్రవర్తిస్తుందా, లేక చిరాకు పుట్టిస్తుందా అనేది ఆధారపడుతుంది.

---

9. Step – "దగ్గరలో ఉన్నది ముందు" – సహజమైన ఆలోచన

మనం 5వ అంతస్తులో ఉన్నాం. `stops` lo 3 మరియు 9 ఉన్నాయి. ఏది ముందు?

సహజమైన జవాబు: **దగ్గరలో ఉన్నది.** 3 అనేది 2 అంతస్తుల దూరం, 9 అనేది 4. కాబట్టి 3కి వెళ్తాం. ఇది logical గా అనిపిస్తుంది, మరియు రాయడానికి సులభం:

```
// stops lo అన్నిటినీ చూసి, ప్రస్తుత అంతస్తుకి అతి దగ్గరిదాన్ని ఎంచుకోవడం
#chooseDirection() {
  if (this.stops.size === 0) { this.direction = Direction.IDLE; return; }

  const nearest = [...this.stops].reduce((a, b) =>
    Math.abs(b - this.currentFloor) < Math.abs(a - this.currentFloor) ? b : a);

  this.direction = nearest > this.currentFloor ? Direction.UP : Direction.DOWN;
}

step() {
  this.#chooseDirection();                        // ప్రతి tick lo మళ్ళీ ఆలోచించడం
  if (this.direction === Direction.IDLE) return;
  this.currentFloor += this.direction;
  this.stops.delete(this.currentFloor);
}
```

ఇది ఏం చేస్తుంది: ప్రతి tick lo అన్ని stops ని చూసి, అతి దగ్గరిదాని వైపు ఒక అడుగు వేస్తుంది.

సులభమైన case lo ఇది బాగానే పనిచేస్తుంది. 0వ అంతస్తులో ఉండి 5, 1, 6 నొక్కితే:

```
ప్రయాణం: 0 → 1 → 2 → 3 → 4 → 5 → 6
మొత్తం ticks: 6
```

ఒక్క వృథా కదలిక కూడా లేదు. కాబట్టి ఇది సరైనదే అనిపిస్తుంది – **ఒక నిర్దిష్ట పరిస్థితి వచ్చేవరకు.**

---

10. రెండో విరుపు – starvation ని కళ్ళతో చూడటం

ఇప్పుడు ఒక నిజమైన ఉదయం traffic ని ఊహించుకుందాం.

Lift **5వ అంతస్తులో** ఉంది. **14వ అంతస్తులో** ఒకరు ఎదురుచూస్తున్నారు – వాళ్ళు చాలా సేపటి నుంచి ఎదురుచూస్తున్నారు. ఈలోగా, మధ్య అంతస్తుల్లో (4, 6) జనం అడపాదడపా బటన్లు నొక్కుతూనే ఉన్నారు.

```
const e = new Elevator('A');
e.currentFloor = 5;
e.addStop(14);                              // 14లో ఒకరు ఎదురుచూస్తున్నారు

const traffic = { 1: 4, 3: 6, 5: 4, 7: 6 };  // tick → ఎవరో దగ్గరలో నొక్కారు
const path = [e.currentFloor];

for (let t = 1; t <= 12; t++) {
  if (traffic[t]) e.addStop(traffic[t]);
  e.step();
  path.push(e.currentFloor);
}

console.log('ప్రయాణం:', path.join(' → '));
console.log('12 ticks తర్వాత 14వ అంతస్తు ఇంకా ఎదురుచూస్తోందా?', e.stops.has(14));
```

```
ప్రయాణం: 5 → 4 → 5 → 6 → 7 → 6 → 5 → 4 → 5 → 6 → 7 → 8 → 9
12 ticks తర్వాత 14వ అంతస్తు ఇంకా ఎదురుచూస్తోందా? true
```

రెండో విరుపు – ఇదే STARVATION

ఆ ప్రయాణాన్ని మళ్ళీ చూడండి: **5 → 4 → 5 → 6 → 7 → 6 → 5 → 4 → 5 → 6 → 7**. Lift పైకి కిందికి ఊగుతోంది. ప్రతిసారి ఎవరో దగ్గరలో నొక్కుతున్నారు, ప్రతిసారి "దగ్గరలో ఉన్నది ముందు" నియమం దాన్ని తిప్పేస్తోంది.

మరియు **14వ అంతస్తులో ఉన్న మనిషి ఇంకా ఎదురుచూస్తూనే ఉన్నారు.** వాళ్ళు మొదటి request ఇచ్చారు, కానీ చివరి వరకు సేవ పొందలేదు. ఇదే **starvation** – ఒక request శాశ్వతంగా వాయిదా పడటం.

ఇది కేవలం ఒక సైద్ధాంతిక సమస్య కాదు. మీరు ఎప్పుడైనా lift కోసం చాలాసేపు ఎదురుచూసి, "ఇది నా అంతస్తు దాటి ఎందుకు వెళ్ళిపోతోంది?" అని చిరాకు పడ్డారా – అది సరిగ్గా ఇదే.

అసలు తప్పు ఎక్కడ ఉంది?

ఒక క్షణం ఆగి ఆలోచిద్దాం. మన `#chooseDirection()` తప్పు లేదు – అది సరిగ్గా అతి దగ్గరిదాన్ని ఎంచుకుంటోంది. **తప్పు ఏమిటంటే – మనం ప్రతి tick lo మళ్ళీ ఆలోచిస్తున్నాం.**

ఒక నిజమైన lift ఇలా ప్రవర్తించదు. అది ఒక దిక్కులో బయలుదేరితే, **ఆ దిక్కులో పని ఉన్నంతవరకు అటే వెళ్తుంది**. మధ్యలో ఎవరు నొక్కినా దాని దిక్కు మారదు. ఆ దిక్కులో పని అయిపోయాకే అది తిరుగుతుంది.

ఇదే **commitment** అనే ఆలోచన. దిక్కుని ఒక *momentary decision* గా కాకుండా, ఒక *commitment* గా చూడటం.

11. Step – LOOK algorithm – మరియు అదే traffic మీద ఫలితం

ఈ commitment ఆలోచనకి ఒక పేరు ఉంది: **LOOK algorithm**. (Disk scheduling నుంచి వచ్చినది – disk head కూడా సరిగ్గా ఇదే సమస్య ఎదుర్కొంటుంది.)

నియమం ఒక్క వాక్యం:

> **ప్రస్తుత దిక్కులో ఇంకా ఏదైనా stop ఉందా? ఉంటే అటే వెళ్ళు. లేకపోతేనే తిరుగు.**

దీన్ని code lo పెట్టాలంటే మనకి ఒక కొత్త ప్రశ్న అడిగే ability కావాలి – *"నా ముందు ఏమైనా ఉందా?"*

```
// నేను వెళ్తున్న దిక్కులో, నా కంటే ముందు ఏదైనా stop ఉందా?
#hasStopAhead() {
  return [...this.stops].some((f) =>
    this.direction === Direction.UP ? f > this.currentFloor
                                    : f < this.currentFloor);
}
```

ఇప్పుడు `step()` ని మారుద్దాం. తేడా ఒక్క పంక్తిలోనే ఉంది, కానీ behaviour పూర్తిగా మారుతుంది:

```
step() {
  if (this.stops.size === 0) { this.direction = Direction.IDLE; return; }

  // ఇక్కడే మొత్తం తేడా: దిక్కు ఎప్పుడు మారుతుంది?
  // ఇప్పుడు ఖాళీగా ఉంటే, లేదా ఈ దిక్కులో ఇక పని లేకపోతే – అప్పుడే.
  if (this.direction === Direction.IDLE || !this.#hasStopAhead()) {
    this.#chooseDirection();
  }

  this.currentFloor += this.direction;
  this.stops.delete(this.currentFloor);
}
```

పాత version: ప్రతి tick lo `#chooseDirection()`. **కొత్త version:** `#chooseDirection()` **అరుదుగా** – కేవలం ఖాళీగా ఉన్నప్పుడు, లేదా ఈ దిక్కులో పని అయిపోయినప్పుడు.

ఇప్పుడు **సరిగ్గా అదే traffic** ని మళ్ళీ నడుపుదాం:

```
ప్రయాణం: 5 → 6 → 7 → 8 → 9 → 10 → 11 → 12 → 13 → 14 → 13 → 12 → 11
14వ అంతస్తు ఇంకా ఎదురుచూస్తోందా? false
మిగిలిన stops: [ 4, 6 ]
```

ఒకే TRAFFIC, రెండు నియమాలు · తేడాని కళ్ళతో చూడటం

5వ అంతస్తు నుంచి మొదలు · 14 ఎదురుచూస్తోంది · మధ్యలో 4, 6 వద్ద జనం నొక్కుతూనే ఉన్నారు

"దగ్గరలో ఉన్నది ముందు" – 12 ticks తర్వాత

5 → 4 → 5 → 6 → 7 → 6 → 5 → 4 → 5 → 6 → 7 → 8 → 9

Lift మధ్యలోనే ఊగుతోంది · దిక్కు ఐదుసార్లు మారింది

14వ అంతస్తు: ఇంకా ఎదురుచూస్తోంది ✗

LOOK – సరిగ్గా అదే traffic, 12 ticks

5 → 6 → 7 → 8 → 9 → 10 → 11 → 12 → 13 → 14 → 13 → 12 → 11

ఒక్కసారే దిక్కు మారింది – 14 చేరాక, పైన ఇక పని లేదు కాబట్టి

14వ అంతస్తు: tick 9 lo సేవ పొందింది ✓ · 4, 6 తిరిగి వస్తూ తీసుకుంటుంది

రెండూ 12 ticks నడిచాయి. తేడా వేగంలో కాదు – **న్యాయంలో**.

ఇక్కడ ఒక ముఖ్యమైన నిజాయితీ

LOOK **ఎక్కువ మందికి వేగంగా సేవ చేయదు**. రెండు versions ఒకే 12 ticks నడిచాయి. మరి తేడా ఏమిటి?

తేడా **న్యాయం (fairness)** మరియు **ఊహించగలగడం (predictability)**.

- LOOK lo ఏ request కూడా శాశ్వతంగా వాయిదా పడదు. Lift ఒక sweep పూర్తిచేసి తప్పక తిరిగి వస్తుంది – కాబట్టి **maximum wait time కి ఒక హద్దు ఉంటుంది**.
- Lift lo ఉన్న ప్రయాణికులు వెనక్కి లాగబడరు. పైకి వెళ్తున్న lift అకస్మాత్తుగా కిందికి తిరగడం – అది ప్రయాణికులకి చాలా చిరాకు.

> **Interview lo ఇది చెప్పండి:** *"LOOK ఎప్పుడూ తక్కువ మొత్తం ticks ఇవ్వదు. కొన్ని సందర్భాల్లో nearest-first వేగంగా ఉంటుంది. కానీ nearest-first lo starvation కి హద్దు లేదు, LOOK lo ఉంది. Lift systems lo మనం optimise చేసేది సగటు wait time కంటే* ***maximum*** *wait time – ఎందుకంటే 3 నిమిషాలు ఎదురుచూసిన ఒక మనిషి, 10 సెకన్లు ఎదురుచూసిన పదిమంది కంటే ఎక్కువ కోపంగా ఉంటారు."* – ఈ చివరి వాక్యం ఒక product ఆలోచన, మరియు interviewer దాన్ని గుర్తిస్తాడు.

---

12. Step – తలుపు, మరియు అది ఒక tick ఎందుకు తినాలి

మన lift ఇప్పుడు సరిగ్గా కదులుతోంది. కానీ ఒక సమస్య మిగిలి ఉంది, మరియు అది subtle.

ప్రస్తుత `step()` lo – lift 3వ అంతస్తు చేరుతుంది, `stops.delete(3)` జరుగుతుంది, మరియు **అదే tick ముగుస్తుంది**. తర్వాతి tick lo అది 4కి కదులుతుంది.

మరి మనుషులు ఎప్పుడు ఎక్కుతారు? **ఎక్కలేరు.** Lift ఆగనే లేదు.

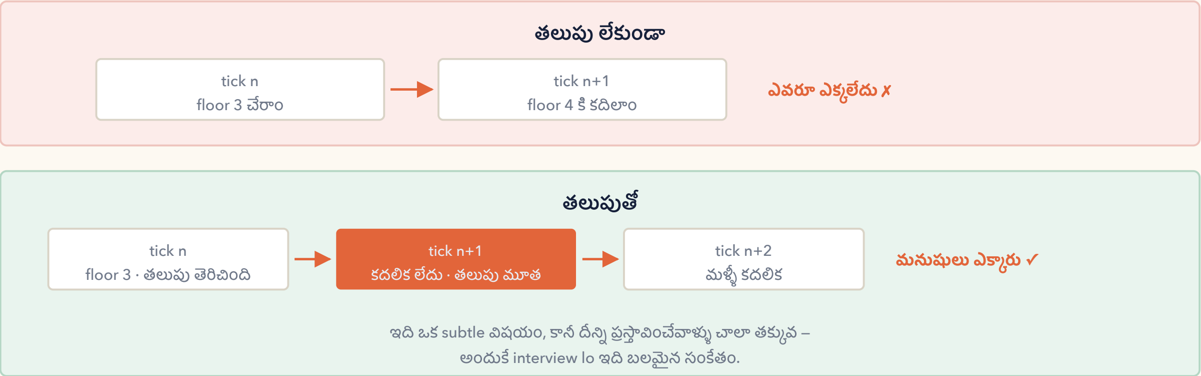
మన model lo "ఒక tick = ఒక అంతస్తు కదలిక". కాబట్టి "ఆగడం" అనేది కూడా **ఒక tick తీసుకోవాలి** – లేకపోతే అది ఆగినట్టే కాదు.

```
  step() {
    // తలుపు తెరిచి ఉంటే — ఈ tick మొత్తం దానికే. కదలిక లేదు.
    if (this.doorOpen) {
      this.doorOpen = false;
      this.#chooseDirection();     // తలుపు మూశాక మళ్ళీ ఆలోచించే అవకాశం
      return;
    }

    if (this.stops.size === 0) { this.direction = Direction.IDLE; return; }
    if (this.direction === Direction.IDLE || !this.#hasStopAhead())
this.#chooseDirection();

    this.currentFloor += this.direction;

    // delete() true తిరిగి ఇస్తే — ఇది మనం ఆగాల్సిన అంతస్తు
    if (this.stops.delete(this.currentFloor)) this.doorOpen = true;
  }
```

ఇది ఏం చేస్తుంది: ఒక stop చేరినప్పుడు `doorOpen = true` అవుతుంది. తర్వాతి `step()` lo lift కదలదు – తలుపు మూసి, ఆ tick ముగుస్తుంది. ఆ తర్వాతి tick నుంచే మళ్ళీ కదలిక.

తలుపు ఒక TICK ఎందుకు తినాలి · రెండు TIMELINES

Lift 3వ అంతస్తు చేరుతోంది – తలుపు లేకుండా, తలుపుతో



తలుపు మూశాక `#chooseDirection()` ఎందుకు పిలుస్తున్నాం?

ఇది ఒక subtle, కానీ ముఖ్యమైన వివరం. తలుపు తెరిచి ఉన్న ఆ tick lo – **ఎవరో లోపలికి వచ్చి కొత్త floor నొక్కి ఉండొచ్చు**. అంటే `stops` మారి ఉండొచ్చు. కాబట్టి తలుపు మూసిన వెంటనే, కదలడానికి ముందు, ఒకసారి మళ్ళీ ఆలోచించడం సరైనది.

ఇది లేకపోతే ఏమవుతుంది? Lift పాత దిక్కులో ఒక అంతస్తు కదిలి, తర్వాత కొత్త stop ని గమనిస్తుంది. ఒక వృథా కదలిక. చిన్న bug, కానీ నిజమైనది.

Part 4 — ఒక lift పూర్తి

> ఇప్పుడు ముక్కలన్నీ కలుపుదాం. ఇది కొత్త code కాదు — మీరు ఇప్పటికే ప్రతి పంక్తినీ చూశారు. ఒకే చోట చూడటమే కొత్త.

---

13. పూర్తి `Elevator` class

```
const Direction = Object.freeze({ UP: 1, IDLE: 0, DOWN: -1 });

class Elevator {
  #stops = new Set();                      // బయటివాళ్ళు నేరుగా మార్చకూడదు

  constructor(id, { minFloor = 0, maxFloor = 14, capacity = 8 } = {}) {
    Object.assign(this, { id, minFloor, maxFloor, capacity });
    this.currentFloor = minFloor;
    this.direction = Direction.IDLE;
    this.doorOpen = false;
    this.load = 0;
  }

  get isFull() { return this.load >= this.capacity; }
  get stops()  { return [...this.#stops]; }      // చదవడానికి copy, మార్చడానికి కాదు

  // ఒక stop చేర్చడం. Car call, hall call — రెండూ ఇక్కడికే వస్తాయి.
  addStop(floor) {
    if (floor < this.minFloor || floor > this.maxFloor)
      throw new Error(`INVALID_FLOOR: ${floor}`);

    if (floor === this.currentFloor) { this.doorOpen = true; return; }   // ఇక్కడే ఉన్నాం

    this.#stops.add(floor);
    if (this.direction === Direction.IDLE) this.#chooseDirection();      // నిద్ర లేపడం
  }

  // LOOK యొక్క గుండె: ఈ దిక్కులో ఇంకా పని ఉందా?
  #hasStopAhead() {
    return [...this.#stops].some((f) =>
      this.direction === Direction.UP ? f > this.currentFloor
                                      : f < this.currentFloor);
  }

  // దిక్కు మార్చాల్సి వచ్చినప్పుడు మాత్రమే — అతి దగ్గరి stop వైపు
  #chooseDirection() {
    if (this.#stops.size === 0) { this.direction = Direction.IDLE; return; }
    const nearest = [...this.#stops].reduce((a, b) =>
      Math.abs(b - this.currentFloor) < Math.abs(a - this.currentFloor) ? b : a);
    this.direction = nearest > this.currentFloor ? Direction.UP : Direction.DOWN;
  }
```

```
  // ఒక యూనిట్ సమయం. ఇదే ఈ class యొక్క ఏకైక "గడియారం".
  step() {
    if (this.doorOpen) { this.doorOpen = false; this.#chooseDirection(); return; }
    if (this.#stops.size === 0) { this.direction = Direction.IDLE; return; }
    if (this.direction === Direction.IDLE || !this.#hasStopAhead())
this.#chooseDirection();

    this.currentFloor += this.direction;
    if (this.#stops.delete(this.currentFloor)) this.doorOpen = true;
  }

  toString() {
    const arrow = { 1: '↑', 0: '·', '-1': '↓' }[this.direction];
    return `${this.id}@${this.currentFloor}${arrow}${this.doorOpen ? '[తలుపు]' :
''}`;
  }
}
```

Steps నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§5	`id`, `currentFloor`	మౌలిక state
§7 (విరుపు)	`target` → `#stops`	ఒకే గమ్యం సరిపోదని తెలిసింది
§8	`Direction`	Set వచ్చాక "ఎటు?" అనేది స్పష్టం కాదు
§11 (LOOK)	`#hasStopAhead()`	Starvation ని ఆపడానికి
§12	`doorOpen`	ఆగడం కూడా సమయం తీసుకోవాలి
ఇక్కడ	`minFloor`, `maxFloor`, `capacity`, `load`	§4 lo అడిగిన constraints

ఇక్కడ చేరిన కొత్తవి రెండే: **range check** (`INVALID_FLOOR`) మరియు **capacity** (`isFull`). రెండూ §4 lo interviewer చెప్పిన constraints నుంచి నేరుగా వచ్చాయి.

`#stops` private ఎందుకు, మరియు `get stops()` ఒక copy ఎందుకు ఇస్తోంది? ఎందుకంటే బయటివాళ్ళు `lift.stops.add(99)` అని రాస్తే – అది `addStop()` lo ఉన్న range check ని, నిద్ర లేపే logic ని పూర్తిగా దాటవేస్తుంది. Copy ఇవ్వడం వల్ల చదవొచ్చు కానీ మార్చలేరు. **Class యొక్క నియమాలని దాటవేసే దారి ఉండకూడదు** – ఇదే encapsulation యొక్క నిజమైన ఉద్దేశం.

---

14. Invariants – ఎప్పుడూ నిజమై ఉండాల్సినవి

ఒక class సరిగ్గా ఉందో లేదో చెప్పే మంచి పద్ధతి – దాని **invariants** రాయడం. Invariant అంటే "ఈ class ఏ state lo ఉన్నా, ఇది ఎప్పుడూ నిజం" అనే statement.

మన `Elevator` కి నాలుగు:

Invariant	ఎందుకు ఇది ముఖ్యం
`minFloor ≤ currentFloor ≤ maxFloor`	Lift భవనం దాటి వెళ్ళకూడదు. `addStop` lo తనిఖీ దీన్ని కాపాడుతుంది
`doorOpen === true` అయితే ఆ tick lo కదలిక లేదు	తలుపు తెరిచి ఉండగా కదలడం physically తప్పు
`#stops.size === 0` అయితే `direction === IDLE`	పని లేకుండా దిక్కు చూపించడం అర్థరహితం
`currentFloor` ఎప్పుడూ `#stops` lo ఉండదు	చేరిన వెంటనే తీసేస్తాం – లేకపోతే అక్కడే ఇరుక్కుపోతుంది

> **Interview lo ఇది ఒక బలమైన కదలిక.** Code రాసి ముగించాక, *"నా design lo ఎప్పుడూ నిజమై ఉండాల్సిన కొన్ని విషయాలు ఉన్నాయి – వాటిని చెప్తాను"* అని ఈ నాలుగు చెప్పండి. ఇది మీరు కేవలం code రాయట్లేదని, ఒక **model** గురించి ఆలోచిస్తున్నారని చూపిస్తుంది. చాలా తక్కువ మంది ఇది చేస్తారు.

Part 5 – ఒకటి నుంచి నాలుగు lifts కి

> ఇప్పటివరకు ఒక lift తనని తాను నడుపుకోవడం చూశాం (§1 lo చెప్పిన **మొదటి సమస్య**). ఇప్పుడు **రెండో సమస్య** – నాలుగు lifts ఉన్నప్పుడు, ఒక కొత్త hall call ని ఎవరికి ఇవ్వాలి?

15. కొత్త సమస్య – ఈ call ని ఎవరికి ఇవ్వాలి?

ఒక దృశ్యం. **6వ అంతస్తులో** ఒకరు **పైకి** వెళ్ళాలని నొక్కారు. నాలుగు lifts ఉన్నాయి:

Lift	ఇప్పుడు ఎక్కడ	ఏం చేస్తోంది
A	5వ అంతస్తు	**పైకి** వెళ్తోంది
B	7వ అంతస్తు	**కిందికి** వెళ్తోంది
C	4వ అంతస్తు	ఖాళీగా నిలబడి ఉంది
D	**6వ అంతస్తు**	కానీ **నిండిపోయింది**

ఒక క్షణం ఆగి, మీరే ఆలోచించండి – ఎవరిని పంపుతారు?

D కాదు – అది సరిగ్గా అదే అంతస్తులో ఉంది, కానీ నిండిపోయింది. తలుపు తెరిచినా ఎవరూ ఎక్కలేరు. **దూరం సున్నా అయినా, అది పనికిరాదు.**

B కాదు – అది 7లో ఉంది, దూరం కేవలం 1. కానీ అది **కిందికి** వెళ్తోంది. ఇప్పుడు అది 6 దగ్గర ఆగితే, పైకి వెళ్ళాలనుకునే మనిషి కిందికి వెళ్ళే lift ఎక్కాలి. అది తప్పు.

A లేదా C. A అనేది 5లో ఉంది, పైకి వెళ్తోంది – అంటే 6 **దాని దారిలోనే** ఉంది. C అనేది 4లో ఖాళీగా ఉంది, 2 అంతస్తులు రావాలి. **A మేలు.**

ఇక్కడ గమనించాల్సినది

మనం "దగ్గరి lift" ని ఎంచుకోలేదు. D దూరం **సున్నా**, B దూరం **1** – కానీ మనం దూరం **1** ఉన్న A ని ఎంచుకున్నాం. అంటే **దూరం ఒక్కటే సరిపోదు**.

ఈ నిర్ణయంలో మూడు విషయాలు కలిశాయి: **దూరం**, **దిక్కు** మరియు **availability**. ఈ మూడింటినీ ఒకే సంఖ్యగా మార్చగలిగితే – అప్పుడు "అతి తక్కువ సంఖ్య ఉన్నవాడిని ఎంచుకో" అని చెప్పొచ్చు. ఆ సంఖ్యనే **cost** అంటారు.

16. Step – cost function ని ఒక్కో పదం చొప్పున కట్టడం

ఒక cost function ని ఒకేసారి రాయడం కష్టం. కాబట్టి **ఒక్కో నియమం చొప్పున** కడదాం – పైన మనం చేసిన ఆలోచననే code గా మారుద్దాం.

నియమం 1 – నిండిన lift పనికిరాదు.

```
if (e.isFull) return Infinity;
```

`Infinity` ఎందుకు, `-1` కాదు? ఎందుకంటే మనం **అతి తక్కువ cost** ఉన్నదాన్ని ఎంచుకుంటాం. `Infinity` అంటే "ఇది ఎప్పటికీ గెలవదు" – మరియు ఇది మిగతా code lo ఒక్క `if` కూడా అవసరం లేకుండా పనిచేస్తుంది. `-1` అయితే అది అతి తక్కువ అయి గెలిచేస్తుంది!

నియమం 2 – basic గా, దూరమే cost.

```
const distance = Math.abs(e.currentFloor - floor);
```

నియమం 3 – ఖాళీ lift కి దూరమే సరిపోతుంది.

```
if (e.direction === Direction.IDLE) return distance;
```

ఖాళీ lift ఎటైనా వెళ్ళగలదు, కాబట్టి దిక్కు గురించి ఆలోచించాల్సిన పని లేదు.

నియమం 4 – కదులుతున్న lift: అది నా దారిలో ఉందా?

ఇక్కడ రెండు షరతులు ఒకేసారి నిజం కావాలి:

- Lift **నా వైపు** కదులుతోందా? (5లో ఉండి పైకి వెళ్తే, 6 దాని వైపే)
- Lift దిక్కు **నా దిక్కుతో** సరిపోతుందా? (నేను పైకి వెళ్ళాలి, lift కూడా పైకి)

```
const movingToward = Math.sign(floor - e.currentFloor) === e.direction;
if (movingToward && e.direction === direction) return distance;
```

నియమం 5 – మిగతా అన్ని సందర్భాల్లో, పెద్ద penalty.

```
return distance + 100;
```

`100` ఎందుకు? ఎందుకంటే మన భవనంలో **15 అంతస్తులే** ఉన్నాయి. అంటే maximum దూరం 14. కాబట్టి 100 అనే penalty – *ఎంత దగ్గరున్నా, తప్పు దిక్కులో ఉన్న lift ఎప్పుడూ సరైన దిక్కులో ఉన్న lift ని ఓడించదు* అని హామీ ఇస్తుంది.

ఇది ఒక magic number, మరియు interview lo దీన్ని మీరే ఒప్పుకోవడం మంచిది: *"ఇక్కడ 100 అనేది ఒక magic number. నిజ system lo ఇది `maxFloor - minFloor + 1` గా ఉండాలి, అప్పుడు భవనం ఎంత*

ఎత్తైనా నియమం వెక్కుచెదరదు." – సొంత code lo ఒక బలహీనతని మీరే చూపించడం ఎప్పుడూ బలమైన కదలిక.

పూర్తి cost function

```
function cost(e, floor, direction) {
  if (e.isFull) return Infinity;                              // 1

  const distance = Math.abs(e.currentFloor - floor);          // 2
  if (e.direction === Direction.IDLE) return distance;        // 3

  const movingToward = Math.sign(floor - e.currentFloor) === e.direction;
  if (movingToward && e.direction === direction) return distance; // 4

  return distance + 100;                                      // 5
}
```

ఇప్పుడు §15 lo ఉన్న సరిగ్గా అదే పరిస్థితిని దీనితో నడుపుదాం:

```
Hall call: floor 6, పైకి
  A (5లో, పైకి వెళ్తోంది) → cost 1
  B (7లో, కిందికి వెళ్తోంది) → cost 101
  C (4లో, ఖాళీగా ఉంది) → cost 2
  D (6లోనే, కానీ నిండింది) → cost Infinity
```

మనం మనసులో చేసిన ఆలోచన, ఇప్పుడు సంఖ్యలుగా మారింది. **A గెలిచింది** (cost 1). D సరిగ్గా అదే అంతస్తులో ఉన్నా `Infinity`. B కేవలం 1 అంతస్తు దూరంలో ఉన్నా 101 – ఎందుకంటే అది తిరిగి రావాలి.

> **Interview lo ఈ పట్టికని గీయండి.** Cost function ని వివరించడానికి ఇది అత్యుత్తమ మార్గం – మీరు నియమాలు చెప్పడం కాదు, **సంఖ్యలు చూపించడం**.

---

17. Step – దాన్ని Strategy గా బయట పెట్టడం, మరియు ఎందుకు

ఇప్పుడు ఒక design నిర్ణయం, మరియు ఇదే ఈ problem lo **అత్యంత ముఖ్యమైనది**.

ఆ `cost` function ని ఎక్కడ పెట్టాలి? రెండు ఎంపికలు:

ఎంపిక 1 – `ElevatorSystem` లోపల ఒక method గా. సులభం, తక్కువ code.

ఎంపిక 2 – ఒక వేరే class గా, బయట నుంచి inject చేయడం. ఒక class ఎక్కువ.

ఏది సరైనది? దీనికి జవాబు చెప్పే ప్రశ్న ఒక్కటే: **"ఇది మారుతుందా?"**

మరియు జవాబు **ఖచ్చితంగా అవును**. ఎందుకంటే §4 lo interviewer ని మనం అడిగాం – *"దేన్ని optimise చేయాలి?"* ఆ ప్రశ్నకి **పలు సరైన జవాబులు** ఉన్నాయి:

- **Wait time** తగ్గించాలంటే → ఇప్పుడు రాసిన nearest-car
- **Energy** ఆదా చేయాలంటే → ఇప్పటికే కదులుతున్న lift కే ఇవ్వాలి, ఖాళీ lift ని లేపకూడదు
- **రాత్రి వేళ** → ఒకే lift నడిపి మిగతావాటిని నిద్రపుచ్చాలి
- **ఉదయం rush** → చాలా lifts ని ground floor lo ఉంచాలి
- **అగ్ని ప్రమాదం** → అన్నీ ground కి, కొత్త calls తిరస్కరణ

అంటే scheduling అనేది **మారే భాగం**. మరియు మారే భాగాన్ని బయటకు పెట్టడమే **Strategy pattern**.

```
// ఒప్పందం (contract): ఏ strategy అయినా ఈ ఒక్క method ఇవ్వాలి
class SchedulingStrategy {
  pick(elevators, floor, direction) {
    throw new Error('subclass ఇది implement చేయాలి');
  }
}

class NearestCarStrategy extends SchedulingStrategy {
  #cost(e, floor, direction) {
    if (e.isFull) return Infinity;
    const distance = Math.abs(e.currentFloor - floor);
    if (e.direction === Direction.IDLE) return distance;
    const movingToward = Math.sign(floor - e.currentFloor) === e.direction;
    if (movingToward && e.direction === direction) return distance;
    return distance + 100;
  }

  pick(elevators, floor, direction) {
    let best = null, bestCost = Infinity;
    for (const e of elevators) {
      const c = this.#cost(e, floor, direction);
      if (c < bestCost) { bestCost = c; best = e; }
    }
    return bestCost === Infinity ? null : best;   // అందరూ నిండారు
  }
}
```

ఆ చివరి పంక్తిని గమనించండి – RETURN NULL

అందరూ నిండిపోతే **ఏమీ చేయలేం**. అప్పుడు ఏం చేయాలి? మూడు ఎంపికలు: (1) `null` తిరిగి ఇచ్చి caller నిర్ణయించనివ్వడం, (2) exception విసరడం, (3) ఏదో ఒక lift కి బలవంతంగా ఇవ్వడం.

మూడోది తప్పు – నిండిన lift కి ఇవ్వడం అంటే ఆ మనిషి ఎదురుచూసి, lift వచ్చి, ఎక్కలేక, మళ్ళీ నొక్కాలి. రెండోది కూడా సరికాదు – ఇది ఒక *exceptional* పరిస్థితి కాదు, ఇది *predictable* పరిస్థితి (ఉదయం rush lo సాధారణం). **కాబట్టి `null`.** Caller దాన్ని queue lo పెట్టుకుని తర్వాత మళ్ళీ ప్రయత్నించవచ్చు.

ఇలాంటి చిన్న నిర్ణయాలని **బయటికి చెప్పడం** – ఇదే interview lo మిమ్మల్ని వేరు చేస్తుంది.

ఇప్పుడు కొత్త strategy రాయడం ఎంత సులభమో చూడండి

```
// "ఖాళీ lift ని లేపొద్దు" – energy ఆదా strategy
class EnergySavingStrategy extends SchedulingStrategy {
  pick(elevators, floor, direction) {
```

```
    const moving = elevators.filter((e) => e.direction !== Direction.IDLE &&
!e.isFull);
    const pool = moving.length ? moving : elevators.filter((e) => !e.isFull);
    return pool.length
      ? pool.reduce((a, b) =>
          Math.abs(b.currentFloor - floor) < Math.abs(a.currentFloor - floor) ? b :
a)
      : null;
  }
}
```

`Elevator` **class lo ఒక్క అక్షరం కూడా మారలేదు.** `ElevatorSystem` **lo కూడా మారదు.** ఇదే Open/Closed Principle – పొడిగించడానికి తెరిచి, మార్చడానికి మూసి. మరియు ఇది ఒక నినాదం కాదు; పైన మీరు దాన్ని *జరగడం* చూశారు.

---

18. Step – `ElevatorSystem` – రెండు call రకాలు ఇక్కడ కలుస్తాయి

చివరి ముక్క. §3 lo మనం hall call, car call అని రెండు రకాలు వేరుచేశాం. ఇప్పుడు ఆ తేడా నేరుగా **రెండు వేర్వేరు methods** గా కనిపిస్తుంది:

```
class ElevatorSystem {
  constructor(elevators, strategy) {
    this.elevators = elevators;
    this.strategy = strategy;              // inject – hardcode కాదు
  }

  // బయటి బటన్: ఏ lift అనేది మనం నిర్ణయిస్తాం
  hallCall(floor, direction) {
    const e = this.strategy.pick(this.elevators, floor, direction);
    if (!e) return null;                   // అందరూ నిండారు
    e.addStop(floor);
    return e.id;                           // "B వస్తోంది" అని చెప్పడానికి
  }

  // లోపలి బటన్: lift ఇప్పటికే తెలుసు, scheduling అవసరం లేదు
  carCall(elevatorId, floor) {
    const e = this.elevators.find((x) => x.id === elevatorId);
    if (!e) throw new Error(`NO_SUCH_ELEVATOR: ${elevatorId}`);
    e.addStop(floor);
  }

  step() { this.elevators.forEach((e) => e.step()); }

  toString() { return this.elevators.map(String).join('   '); }
}
```

ఈ రెండు methods ని పక్కపక్కన చూడండి – **§3 lo చేసిన విభజన ఇక్కడ నేరుగా కనిపిస్తోంది:**

- `hallCall` కి **strategy కావాలి**, ఎందుకంటే ఏ lift అనేది తెలియదు. అది ఒక `id` తిరిగి ఇస్తుంది.
- `carCall` కి **strategy అవసరమే లేదు**, ఎందుకంటే lift ఇప్పటికే తెలుసు. అది ఏమీ తిరిగి ఇవ్వదు.

- `hallCall` కి `direction` కావాలి. `carCall` కి **అవసరం లేదు**.

> ఇదే ఒక మంచి design యొక్క లక్షణం: **మొదట్లో చేసిన సరైన విభజన, చివర్లో code ని సహజంగా ఆకారం ఇస్తుంది.** మనం §3 lo ఆ తేడాని గుర్తించకపోతే, ఇక్కడ ఒకే `request()` method lo `if (direction === null)` అనే గందరగోళం ఉండేది.

Part 6 — పూర్తి system

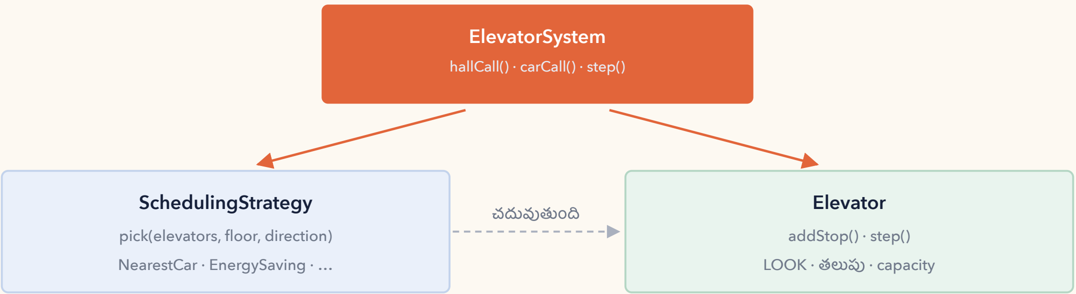
> మూడు classes, మొత్తం ~80 lines. ప్రతి పంక్తికీ ఒక కారణం ఉంది, మరియు ఆ కారణం మీకు తెలుసు.

---

19. మొత్తం code ఒకే చోట

మూడు CLASSES · ఎవరికి ఏమి తెలుసు, ఎవరికి తెలియదు

బాణం అర్థం: "దీని గురించి తెలుసు". తెలియకపోవడం కూడా ఒక design నిర్ణయమే.



ఇక్కడ ముఖ్యమైనది – ఎవరికి ఏమి తెలియదు

Elevator కి strategy గురించి తెలియదు · మిగతా lifts గురించి తెలియదు · system గురించి తెలియదు
Strategy కి lift లోపలి LOOK logic తెలియదు – అది కేవలం floor, direction, isFull చదువుతుంది
ఈ "తెలియకపోవడం" వల్లనే మూడింటిని విడిగా మార్చొచ్చు, విడిగా test చేయొచ్చు

```
const Direction = Object.freeze({ UP: 1, IDLE: 0, DOWN: -1 });

class Elevator {
  #stops = new Set();
  constructor(id, { minFloor = 0, maxFloor = 14, capacity = 8 } = {}) {
    Object.assign(this, { id, minFloor, maxFloor, capacity });
    this.currentFloor = minFloor;
    this.direction = Direction.IDLE;
    this.doorOpen = false;
    this.load = 0;
  }
  get isFull() { return this.load >= this.capacity; }
  get stops()  { return [...this.#stops]; }

  addStop(floor) {
    if (floor < this.minFloor || floor > this.maxFloor)
      throw new Error(`INVALID_FLOOR: ${floor}`);
    if (floor === this.currentFloor) { this.doorOpen = true; return; }
    this.#stops.add(floor);
    if (this.direction === Direction.IDLE) this.#chooseDirection();
  }
  #hasStopAhead() {
    return [...this.#stops].some((f) =>
      this.direction === Direction.UP ? f > this.currentFloor : f <
```

```
this.currentFloor);
  }
  #chooseDirection() {
    if (this.#stops.size === 0) { this.direction = Direction.IDLE; return; }
    const nearest = [...this.#stops].reduce((a, b) =>
      Math.abs(b - this.currentFloor) < Math.abs(a - this.currentFloor) ? b : a);
    this.direction = nearest > this.currentFloor ? Direction.UP : Direction.DOWN;
  }
  step() {
    if (this.doorOpen) { this.doorOpen = false; this.#chooseDirection(); return; }
    if (this.#stops.size === 0) { this.direction = Direction.IDLE; return; }
    if (this.direction === Direction.IDLE || !this.#hasStopAhead())
this.#chooseDirection();
    this.currentFloor += this.direction;
    if (this.#stops.delete(this.currentFloor)) this.doorOpen = true;
  }
  toString() {
    const arrow = { 1: '↑', 0: '·', '-1': '↓' }[this.direction];
    return `${this.id}@${this.currentFloor}${arrow}${this.doorOpen ? '[తలుపు]' :
''}`;
  }
}

class NearestCarStrategy {
  cost(e, floor, direction) {
    if (e.isFull) return Infinity;
    const distance = Math.abs(e.currentFloor - floor);
    if (e.direction === Direction.IDLE) return distance;
    const movingToward = Math.sign(floor - e.currentFloor) === e.direction;
    if (movingToward && e.direction === direction) return distance;
    return distance + 100;
  }
  pick(elevators, floor, direction) {
    let best = null, bestCost = Infinity;
    for (const e of elevators) {
      const c = this.cost(e, floor, direction);
      if (c < bestCost) { bestCost = c; best = e; }
    }
    return bestCost === Infinity ? null : best;
  }
}

class ElevatorSystem {
  constructor(elevators, strategy) { this.elevators = elevators; this.strategy =
strategy; }
  hallCall(floor, direction) {
    const e = this.strategy.pick(this.elevators, floor, direction);
    if (!e) return null;
    e.addStop(floor);
    return e.id;
  }
  carCall(elevatorId, floor) {
    const e = this.elevators.find((x) => x.id === elevatorId);
    if (!e) throw new Error(`NO_SUCH_ELEVATOR: ${elevatorId}`);
    e.addStop(floor);
  }
  step() { this.elevators.forEach((e) => e.step()); }
```

```
  toString() { return this.elevators.map(String).join('   '); }
}
```

20. ఒక పూర్తి simulation – tick by tick

ఇప్పుడు మూడు lifts ని వేర్వేరు అంతస్తుల్లో పెట్టి, రెండు hall calls ఇచ్చి, నడిపి చూద్దాం:

```
const lifts = [new Elevator('A'), new Elevator('B'), new Elevator('C')];
lifts[1].currentFloor = 8;
lifts[2].currentFloor = 12;
const sys = new ElevatorSystem(lifts, new NearestCarStrategy());

console.log('మొదలు:      ', sys.toString());
console.log('hallCall(10, UP)  →', sys.hallCall(10, Direction.UP));
console.log('hallCall(2, UP)   →', sys.hallCall(2, Direction.UP));

for (let t = 1; t <= 6; t++) { sys.step(); console.log(`tick ${t}:      `,
sys.toString()); }

console.log('carCall(B, 14)');
sys.carCall('B', 14);
for (let t = 7; t <= 10; t++) { sys.step(); console.log(`tick ${t}:      `,
sys.toString()); }
```

```
మొదలు:       A@0·   B@8·   C@12·
hallCall(10, UP)  → B
hallCall(2, UP)   → A
tick 1:       A@1↑   B@9↑   C@12·
tick 2:       A@2↑[తలుపు]   B@10↑[తలుపు]   C@12·
tick 3:       A@2·   B@10·   C@12·
tick 4:       A@2·   B@10·   C@12·
tick 5:       A@2·   B@10·   C@12·
tick 6:       A@2·   B@10·   C@12·
carCall(B, 14)
tick 7:       A@2·   B@11↑   C@12·
tick 8:       A@2·   B@12↑   C@12·
tick 9:       A@2·   B@13↑   C@12·
tick 10:       A@2·   B@14↑[తలుపు]   C@12·
```

ఈ output ని పంక్తి పంక్తిగా చదువుదాం

`hallCall(10, UP) → B`. ఎందుకు B? అందరూ ఖాళీగా ఉన్నారు, కాబట్టి cost = దూరం. A నుంచి 10కి దూరం 10, B నుంచి 2, C నుంచి 2. B మరియు C రెండూ 2 – **సమానం**. మన `pick()` lo `c < bestCost` అని ఉంది (`<=` కాదు), కాబట్టి **మొదట కనిపించినవాడు గెలుస్తాడు** – అది B. ఇది ఒక arbitrary ఎంపిక, కానీ **predictable**, మరియు అదే ముఖ్యం.

`hallCall(2, UP) → A`. ఇప్పుడు B 10 వైపు కదులుతోంది (cost 8 + penalty), C ఇంకా ఖాళీ (దూరం 10), A ఖాళీ (దూరం 2). **A గెలిచింది.**

Tick 2 – `A@2↑[తలుపు]` **.** A రెండో అంతస్తు చేరి తలుపు తెరిచింది. అదే tick lo B కూడా 10 చేరి తలుపు తెరిచింది.

Tick 3 – `A@2·` **.** గమనించండి: A **కదలలేదు**. ఇది §12 lo మనం కట్టిన behaviour – తలుపు ఒక tick తింటుంది. తలుపు మూసింది, `stops` ఖాళీ కాబట్టి దిక్కు `IDLE` (`·`) అయింది.

Ticks 4, 5, 6 – ఏమీ జరగలేదు. మూడు lifts ఖాళీగా నిలబడి ఉన్నాయి. పని లేకపోతే కదలవు – సరైనది.

`carCall(B, 14)` **తర్వాత tick 7 నుంచి.** B lo ఎవరో 14 నొక్కారు. Scheduling అవసరం లేదు – B కే వెళ్ళింది. Ticks 7–10 lo B 11 → 12 → 13 → 14 కి కదిలి తలుపు తెరిచింది.

గమనించాల్సినది: tick 8 lo B 12కి చేరింది, **C కూడా అక్కడే ఉంది**. ఏమీ జరగలేదు – సరైనది. మన model lo lifts వేర్వేరు shafts lo ఉన్నాయి. ఒకే shaft lo ఉంటే అది పూర్తిగా వేరే problem (§23 lo చూడండి).

ఈ output ని మీరే నడిపి చూడండి. అచ్చు ఇదే రాకపోతే, ఎక్కడ తేడా వచ్చిందో వెతకండి – ముఖ్యంగా `doorOpen` handling lo. ఆ వెతుకులాట ఈ doc మొత్తం చదవడం కంటే ఎక్కువ నేర్పుతుంది.

Part 7 – Interview lo

> ఇక్కడివరకు మనం **అర్థం చేసుకోవడానికి** నేర్చుకున్నాం. ఇప్పుడు అదే విషయాన్ని **45 నిమిషాల్లో చెప్పడం** ఎలా అనేది.

---

21. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

ఈ doc చదవడానికి గంట పడుతుంది. Interview lo మీకు 45 నిమిషాలే. కాబట్టి **ఏమి చెప్పాలి, ఏమి వదిలేయాలి** అనేది ముఖ్యం.

45 నిమిషాల TIME BUDGET · ఏ దశకి ఎంత

ఈ ratio ముఖ్యం – చాలా మంది మొదటి రెండు అడుగులని దాటవేసి ఓడతారు

5 నిమి	**Clarify** – ముఖ్యంగా hall vs car call తేడాని మీరే లేవనెత్తండి (§3)
3	Nouns → classes · **Floor ని class చేయనని** చెప్పడం (§21 కింద)
5 నిమి	State + LOOK ని **గీయడం** – code కంటే ముందు
20 నిమి – Elevator class	ఇదే మీ ప్రధాన సమయం
7 నిమి	Strategy + System
5 నిమి	Follow-ups, extensibility

మొదటి 13 నిమిషాలు code లేకుండా. ఇది వృథా కాదు – **ఇదే మిమ్మల్ని select చేసే భాగం.**

ఏమి వదిలేయాలి

- `EnergySavingStrategy` **రాయొద్దు.** "కొత్త strategy అంటే కొత్త class, ఉన్నవాటిలో సున్నా మార్పులు" అని **చెప్తే** చాలు. అడిగితేనే రాయండి.
- `toString()` **వదిలేయండి** – అది debugging కోసం, design కోసం కాదు.
- **Capacity, load ని చివర్లో ఒక్క పంక్తిలో** చేర్చండి, మొదట్లో కాదు.
- **Simulation loop రాయొద్దు** – బదులుగా మౌఖికంగా trace చేయండి: *"Lift 5లో ఉంది, 8 నొక్కారు, కాబట్టి..."*

ఏమి తప్పక చెప్పాలి

1. **Hall call vs car call తేడా** (§3) – ఇది మీరే లేవనెత్తాలి, అడిగేవరకు ఆగకూడదు.
2. **Nearest-first ఎందుకు విఫలమవుతుందో** (§10) – starvation అనే పదం వాడండి.
3. **Scheduling ని బయట పెట్టడం** (§17) – మరియు **ఎందుకు** (optimisation లక్ష్యం మారుతుంది).
4. **తలుపు ఒక tick తింటుంది** (§12) – ఇది చిన్నది, కానీ చాలా తక్కువ మంది చెప్తారు.

"Floor ని class చేయరా?" – దీన్ని మీరే లేవనెత్తండి

Nouns → classes చేసేటప్పుడు, requirement lo "floor" అనే పదం ఉంది. కాబట్టి `Floor` class రాయాలా?

> *"Floor ని నేను class చేయట్లేదు. దానికి state లేదు, behaviour లేదు – అది ఒక integer. దానికి class రాయడం అంటే అనవసరమైన abstraction. కానీ ప్రతి floor కి తనదైన లక్షణాలు ఉంటే – access control, floor-specific display, restricted floors – అప్పుడు నేను దాన్ని class చేస్తాను.* ***ఇప్పుడు లేని అవసరం కోసం abstraction సృష్టించను.****"*

ఇదే `Button` కి, `Person` కి కూడా వర్తిస్తుంది. Button అనేది UI; domain model lo దానికి స్థానం లేదు. Person ని system track చేయదు – కేవలం `load` అనే ఒక సంఖ్య చాలు.

---

22. నోటితో చెప్పాల్సిన English script

ఇది కంఠస్థం చేయడానికి కాదు – **లయ** కోసం. ఈ నిర్మాణాన్ని గమనించండి: విభజన → ఒక design నిర్ణయం → **ఎందుకు** → ఒక సొంత లోపాన్ని ఒప్పుకోవడం.

"Before I write anything, let me split this into two problems, because I think they're quite different.

The first is how a single elevator behaves – where it goes next, when it reverses. The second is which elevator should answer a new call. I'll do them in that order.

One thing I want to flag early: there are two kinds of request here, and they carry different information. A **hall call** – someone on floor 5 pressing 'up' – tells me the floor and the direction, but not the destination, and crucially the system has to decide which car goes. A **car call** – someone inside pressing 9 – tells me the destination, and the car is already decided. So those become two different methods with different signatures. If I forced them into one `Request` class, I'd be carrying a null direction around for every car call, and that null would be a lie in my model.

For a single elevator, the obvious approach is to always serve the nearest stop. That's wrong, and I can show you why: if calls keep arriving near the car, it oscillates and a far request can wait forever – that's starvation. So I'll use **LOOK**: keep going in the current direction while there's any stop ahead, and only reverse when there isn't. That doesn't reduce total travel, but it bounds the worst-case wait, and for elevators the worst case is what people actually complain about.

One detail I'd like to get right: stopping has to consume a tick. If the car arrives at a floor and moves on in the same tick, nobody can actually board. So an open door is a tick where the car doesn't move.

For picking a car, I'll score each one – distance, plus a large penalty if it's moving the wrong way, and infinity if it's full. Then take the minimum. I'm going to put that scoring behind a `SchedulingStrategy` interface rather than inline it, because the thing we optimise for genuinely changes: average wait during the day, energy at night, everything-to-the-ground in a fire. A new policy should be a new class, not an edit to the elevator.

One weakness I'd call out in my own code: that penalty is a magic 100. It should be `maxFloor - minFloor + 1`, so the rule holds for any building height."

> ఆ **చివరి paragraph** – సొంత లోపాన్ని మీరే చూపించడం – దీన్ని ఎప్పుడూ వదలొద్దు. ఇది ఒక బలహీనతగా కనిపిస్తుంది, కానీ interviewer దృష్టిలో ఇది **మీ code ని మీరే critically చదవగలరని** రుజువు. అది ఒక senior లక్షణం.

---

23. Follow-ups – మరియు మన design ఎలా తట్టుకుంటుంది

Interviewer ఖచ్చితంగా ఒక కొత్త requirement విసురుతాడు. మన design దానికి ఎలా స్పందిస్తుందో చూద్దాం – **ముఖ్యంగా, ఎన్ని ఉన్న classes మారాలో**.

Follow-up	మన design ఏం చేస్తుంది	మారే classes
"రాత్రి వేళ energy ఆదా చేయాలి"	కొత్త `EnergySavingStrategy` (§17)	**0**
"Express lift – 10 పైన floors మాత్రమే"	`Elevator` కి `servableFloors`; strategy lo ఒక filter	Elevator + Strategy
"అగ్ని ప్రమాదం – అన్నీ ground కి"	`FireModeStrategy` + lifts కి `emergencyRecall()`	**+1 కొత్తది**
"Maintenance mode"	`isAvailable` flag; strategy lo `Infinity`	Elevator lo ఒక field
"ఏ lift వస్తుందో బయట display చూపించాలి"	`hallCall()` ఇప్పటికే `id` తిరిగి ఇస్తోంది (§18)	**0**
"నిజమైన time, ticks కాదు"	పైన ఒక timer, అది `step()` పిలుస్తుంది (§4)	**0** – ఇదే tick-based ఎంచుకున్న లాభం
"ఒక shaft lo రెండు lifts (double-deck)"	ఇది **నిజంగా కష్టం** – కింద చూడండి	పెద్ద మార్పు

ఆ చివరి దానికి నిజాయితీగా జవాబు

Double-deck lifts (ఒకే shaft lo రెండు cars) – ఇక్కడ మన model **నిజంగా విఫలమవుతుంది**, మరియు దాన్ని ఒప్పుకోవడమే సరైన జవాబు:

> *"ఇది నా design ని నిజంగా విరగ్గొడుతుంది. ఇప్పుడు నా `Elevator` తన floor ని independently మార్చుకుంటుంది – అది ఇతర lifts గురించి ఏమీ తెలియకుండా ఉండటమే నా design యొక్క బలం. ఒకే shaft lo రెండు cars ఉంటే, వాటి కదలికల మధ్య ఒక* ***constraint*** *ఉంటుంది – అవి ఒకదానిలో ఒకటి దూరిపోకూడదు. అంటే floor ని మార్చే నిర్ణయం ఇక car ది కాదు, shaft ది. నేను ఒక `Shaft` class ప్రవేశపెట్టి, `step()` ని అక్కడికి తరలించాల్సి వస్తుంది. అది ఒక చిన్న మార్పు కాదు."*

ఈ రకమైన జవాబు – "*ఇది నా design ని విరగ్గొడుతుంది, మరియు ఎక్కడ విరుగుతుందో ఇది*" – ఒక తప్పుడు "అది కూడా సులభమే" కంటే **చాలా బలమైనది**.

మరో రెండు ప్రశ్నలు, తరచుగా వచ్చేవి

"Thread safety గురించి ఏమంటారు?"

> *"JavaScript lo ఇది single-threaded, కాబట్టి `step()` మధ్యలో ఎవరూ దూరలేరు. కానీ Java lo అయితే – `addStop` మరియు `step` ఒకేసారి జరిగితే `stops` Set పాడవుతుంది. అక్కడ నేను elevator కి ఒక lock, లేదా మంచిది – ఒక command queue పెడతాను: బయటి calls queue lo పడతాయి, `step()` వాటిని tick మొదట్లో తీసుకుంటుంది. అప్పుడు lock అవసరమే ఉండదు."*

"దీన్ని ఎలా test చేస్తారు?"

> *"Tick-based కాబట్టి ఇది చాలా సులభం. ఒక lift ని ఒక state lo పెట్టి, `step()` ని n సార్లు పిలిచి, floor ఎక్కడ ఉందో assert చేయొచ్చు – timers అవసరం లేదు, ఎదురుచూపు అవసరం లేదు. §10 lo చూపించిన starvation కూడా ఒక test case గా రాయొచ్చు: ఈ traffic ఇస్తే, 12 ticks తర్వాత floor 14 తప్పక సేవ పొంది ఉండాలి. అదే `NearestFirst` తో fail అవుతుంది, LOOK తో pass అవుతుంది."*

---

24. ఏమి నేర్చుకున్నాం

ఈ doc lo lift గురించి నేర్చుకున్నది కొంచెమే. నిజంగా నేర్చుకున్నవి ఇవి – **ఇవి మిగతా ప్రతి LLD problem కీ వర్తిస్తాయి**:

ఆలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
సమస్యని రెండుగా విడగొట్టడం	"ఒక lift లోపల" vs "lifts మధ్య"	దాదాపు ప్రతి design problem lo
రెండు రకాల inputs ని గుర్తించడం	Hall call vs car call	Booking (search vs confirm), payment (authorise vs capture)
సహజమైన algorithm ముందు, దాని failure తర్వాత	Nearest-first → starvation → LOOK	Cache eviction, rate limiting, scheduling – అన్నిటిలోనూ
మారేదాన్ని బయట పెట్టడం	`SchedulingStrategy`	Pricing, discount, notification channel, retry policy
సమయాన్ని inject చేయడం	`step()`, `Date.now()` కాదు	Rate limiter, TTL cache, session expiry
Invariants ని బయటకి చెప్పడం	§14 lo నాలుగు	ఏ stateful class కైనా
సొంత లోపాన్ని ఒప్పుకోవడం	Magic 100, double-deck	ప్రతి interview lo

ఇక్కడి నుంచి ఎక్కడికి

ఈ doc `LLD_Design_Problems_Telugu.pdf` lo ఉన్న **Problem 02** యొక్క expanded version. ఆ book lo అదే problem **interview నమూనా** గా ఉంటుంది – తక్కువ మాటలు, ఎక్కువ వేగం. రెండూ ఒకే design ని చెప్తాయి; తేడా **వేగంలో**.

సూచించే క్రమం: ముందు ఇది చదివి అర్థం చేసుకోండి. తర్వాత ఆ book lo Problem 02 చదివి, **అదే విషయాన్ని 45 నిమిషాల్లో ఎలా చెప్పాలో** చూడండి. రెండోది మొదటిది లేకుండా చదివితే అది కేవలం గుర్తుపెట్టుకోవడం అవుతుంది.

LOOK వెనక ఉన్న సాధారణ సూత్రాల కోసం – `LLD_Telugu.pdf` §29 (Strategy), §28 (State), మరియు §5 (SOLID, ముఖ్యంగా Open/Closed).

Elevator System – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output నిజంగా `node` lo run చేసి తీసినదే ✅